Building an Enterprise Credit Underwriting Platform

Architected and Engineered by Sasidhar Naram

1. The Business Problem

Commercial lending relies on accurate, timely risk assessments. However, the existing process for onboarding MSME (Micro, Small, and Medium Enterprises) borrowers is fundamentally broken.

1.1 The Existing Process

When a company like Harika Shipping & Logistics applies for a loan, they submit a massive data dump: 30-page audited financial statements (ITRs), 12 months of GST returns, and multi-bank ledgers. A highly-paid Credit Analyst then manually opens these PDFs side-by-side, visually scans for numbers, and copy-pastes them into a sprawling Excel spreadsheet known as the Credit Assessment Memo (CAM).

1.2 The Pain Points

- Speed: This manual process takes 2 to 3 days per borrower.
- Accuracy: Complex calculations (like adjusting Creditor Days by extracting nested Direct Expenses) are prone to human error.
- Risk Blindness: It is impossible for a human to visually scan 10,000 lines of bank statements to calculate an exact "Bounce Rate" or identify fund diversion.
- Auditability: Because rules are applied subjectively by different analysts, there is no centralized, enforceable credit policy.

2. The Solution: CUIS

I built the Credit Underwriting Intelligence Suite (CUIS) to solve this bottleneck. CUIS is an enterprise decision-support platform that transforms fragmented manual tasks into a centralized, automated pipeline. By treating underwriting as a deterministic data problem rather than a subjective human task, CUIS ensures 100% policy compliance, auditability, and speed.

3. Architecture

CUIS is built using Clean Architecture. This was a deliberate choice to separate messy parsing infrastructure from core banking rules.

1. Infrastructure (Parsers): Specialized regex engines extract data from PDFs.
2. Database: Extracted facts are stored in a centralized SQLite database.
3. Reconciliation Service: Data is cross-checked (e.g., Audited Sales vs GST Sales).
4. Policy Engine: Reconciled data is evaluated against configurable mandates.
5. Presentation: The system generates a bank-ready Excel CAM and an HTML Dashboard.

4. Key Design Decisions

4.1 Configurable Rule Engine vs. Machine Learning

I deliberately chose a deterministic Rule Engine over training an ML model. In commercial lending, regulatory bodies require banks to explicitly state *why* a loan was rejected. A black-box ML model cannot easily provide legal justification. A rule engine is 100% auditable.

4.2 Spatial-Aware Regex Parsing

Standard PDF extractors flatten text, destroying the visual hierarchy of an accounting document. I built regex rules that first capture a spatial "Zone" (e.g., everything between "Purchases" and "Gross Profit"), allowing the engine to accurately isolate and sum Direct Expenses regardless of formatting.

5. Challenges Overcome

During development, a critical bug emerged: The Working Capital "Creditor Days" calculation for a logistics company was showing an absurd 19,000+ days. I traced this to the Cost of Sales denominator. The parser was only extracting raw goods "Purchases" (which are near zero for a service company), ignoring millions in "Vessel Charges" and "Godown Rents". I had to rewrite the FinancialParser to dynamically hunt for the "Manufacturing/Direct Expenses" block. This instantly realigned the calculations to the bank's audited standards, bringing the Creditor Days to a compliant 107 Days.

6. Results & Impact

- Time Saved: CAM preparation reduced from 2-3 days to under 2 minutes.

- Accuracy: 100% automated extraction and math execution.

- Risk Visibility: The Banking Engine automatically calculates exact bounce rates and categorizes cash flows across the entire ledger.